FirstOffer

CODING INTERVIEW KIT · 2026 EDITION

24 Must-Practice Coding Problems + 6 Bonus Questions

The patterns that keep showing up in fresher and SDE interviews — explained simply, solved cleanly, and ordered from the boss level down to the warm-ups.

Arrays & Hashing | Linked Lists | Trees & Graphs | Dynamic Programming | Python solutions

30 problems

11 patterns

100% tested code

firstoffer.online

Fresher jobs · Direct apply links · No middlemen

Free to share with friends

START HERE

How to use this kit

1
Read the problem and try it yourself for 20 minutes before looking at the solution.

2
Name the pattern first. The table below maps common clues to the right tool.

3
Re-solve it a week later without notes. Tick it off on the checklist at the end.

SPOT THE PATTERN

If the question says… reach for…

“Find a pair / count things / have I seen this?”	**Hash map or set**
“Longest / shortest contiguous part of a string or array”	**Sliding window**
“The input is sorted” or “O(log n) please”	**Binary search · two pointers**
“Overlapping ranges, meetings, schedules”	**Sort, then sweep**
“Brackets, undo, most recent first”	**Stack**
“Nodes with next pointers”	**Pointer rewiring · dummy node · fast/slow**
“Level by level” or “shortest steps”	**BFS with a queue**
“Regions, connections, copies of a graph”	**DFS / BFS + visited set**
“Do A before B” dependencies	**Topological sort**
“Top k / kth largest / always the smallest”	**Heap**
“Best total / number of ways, built from smaller answers”	**Dynamic programming**

IN THE INTERVIEW ROOM

1 · Repeat the problem back **2 · Ask about edge cases** **3 · Say the brute force**
4 · Improve it out loud **5 · Code cleanly** **6 · Dry-run an example** **7 · State time & space**

CONTENTS

Hardest first, warm-ups last

This kit runs upside down: start with the toughest pattern while your mind is fresh, then lock in the classics.

Level 6 · Two Pointers (Advanced)

01	Trapping Rain Water	Hard

Level 5 · Dynamic Programming

02	House Robber	Medium
03	Coin Change	Medium
04	Word Break	Medium

Level 4 · Heaps, Graphs & Trees

05	Top K Frequent Elements	Medium
06	Course Schedule	Medium
07	Clone Graph	Medium
08	Number of Islands	Medium
09	Lowest Common Ancestor in a BST	Medium
10	Level Order Traversal	Medium
11	Kth Largest Element	Medium

Level 3 · Binary Search

12	Search in Rotated Sorted Array	Medium
13	Binary Search	Easy

Level 2 · Linked Lists & Stacks

14	LRU Cache	Medium / Hard
15	Merge Two Sorted Lists	Easy
16	Reverse a Linked List	Easy
17	Min Stack	Medium
18	Valid Parentheses	Easy

Level 1 · Arrays & Hashing

19	Merge Intervals	Medium
20	3Sum	Medium
21	Product of Array Except Self	Medium
22	Longest Substring Without Repeats	Medium
23	Best Time to Buy and Sell Stock	Easy
24	Two Sum	Easy

★ Bonus · Most-asked fresher questions

B1	Maximum Subarray (Kadane's Algorithm)	Medium
B2	Valid Anagram	Easy
B3	Detect a Cycle in a Linked List	Easy
B4	Missing Number	Easy
B5	Group Anagrams	Medium
B6	Climbing Stairs	Easy

Level 6 · Two Pointers (Advanced)

01 Trapping Rain Water

Two Pointers · Hard

Bars of different heights stand side by side (width 1 each). After it rains, how many units of water stay trapped between them?

EXAMPLE `height = [0,1,0,2,1,0,1,3,2,1,2,1]` → **6**

HOW TO THINK

1. Water above a bar = min(tallest bar on its left, tallest on its right) − its own height.
2. Walk two pointers inwards. The side with the shorter wall is the one that limits the water, so settle that side first.
3. Keep a running maximum for each side and add the difference as you move.

COMPLEXITY

Time O(n)

Space O(1)

```
def trap(height):
    lo, hi = 0, len(height) - 1
    left_wall = right_wall = water = 0
    while lo < hi:
        if height[lo] < height[hi]:
            left_wall = max(left_wall, height[lo])
            water += left_wall - height[lo]
            lo += 1
        else:
            right_wall = max(right_wall, height[hi])
            water += right_wall - height[hi]
            hi -= 1
    return water
```

Edge cases Empty list or fewer than 3 bars → 0. Strictly rising or falling bars hold no water.

Interviewer may ask next Solve it again with a stack. How would you handle a 2-D height map?

02 House Robber

1-D Dynamic Programming · Medium

Each house on a street holds some cash. You can't rob two houses that are next to each other. What's the most cash you can collect?

`EXAMPLE houses = [2,7,9,3,1] → 12 (2 + 9 + 1)`

HOW TO THINK

1. At every house you have two choices: rob it (and add the best total from two houses back) or skip it.
2. You only ever need the last two answers, so two variables replace the whole DP table.

COMPLEXITY

Time O(n)

Space O(1)

```
def rob(houses):
    skip, take = 0, 0          # best total if we skip / take the previous house
    for cash in houses:
        skip, take = max(skip, take), skip + cash
    return max(skip, take)
```

Edge cases No houses → 0. One house → its value.

Interviewer may ask next The houses are in a circle (first and last are neighbours). What changes?

03 Coin Change

Unbounded Knapsack DP · Medium

Given coin values and a target amount, find the fewest coins that add up to the amount. Return −1 if it can't be made.

`EXAMPLE` coins = [1,2,5], amount = 11 → **3 (5 + 5 + 1)**

HOW TO THINK

1. fewest[t] = smallest number of coins that make total t.
2. For each total, try every coin c: fewest[t] = min(fewest[t], fewest[t − c] + 1).
3. Build from 0 up to the amount; anything still 'infinite' is impossible.

COMPLEXITY

Time O(amount × coins)

Space O(amount)

```
def coin_change(coins, amount):
    INF = amount + 1
    fewest = [0] + [INF] * amount
    for total in range(1, amount + 1):
        for c in coins:
            if c <= total:
                fewest[total] = min(fewest[total], fewest[total - c] + 1)
    return -1 if fewest[amount] == INF else fewest[amount]
```

Edge cases amount = 0 → 0 coins. A greedy 'biggest coin first' fails for coins like [1,3,4], amount 6.

Interviewer may ask next Count the number of different ways to make the amount instead.

04 Word Break

String Dynamic Programming · Medium

Can a string be split into a sequence of words that all appear in a given dictionary? Words may be reused.

`EXAMPLE s = "firstoffer", words = ["first","offer"] → True`

HOW TO THINK

1. can[i] tells whether the first i characters can be split.
2. can[end] is true if some earlier cut point start is true and s[start:end] is a dictionary word.
3. Put the dictionary in a set so each lookup is fast.

COMPLEXITY

Time $O(n^2 \cdot L)$ (L = slice cost)

Space O(n)

```
def word_break(s, words):
    vocab = set(words)
    can = [True] + [False] * len(s)    # can[i]: s[:i] splits into words
    for end in range(1, len(s) + 1):
        for start in range(end):
            if can[start] and s[start:end] in vocab:
                can[end] = True
                break
    return can[len(s)]
```

Edge cases Empty string → True. Watch out for greedy matching: "catsandog" looks splittable but isn't.

Interviewer may ask next Return every possible sentence, not just True/False (Word Break II).

05 Top K Frequent Elements

Counting + Bucket Sort · Medium

Return the k numbers that appear most often in an array.

`EXAMPLE nums = [1,1,1,2,2,3], k = 2 → [1, 2]`

HOW TO THINK

1. Count every number with a hash map.
2. A number can appear at most n times, so drop each value into a bucket indexed by its count.
3. Read buckets from the highest count down until you have k numbers.

COMPLEXITY

Time O(n)

Space O(n)

```
def top_k_frequent(nums, k):
    counts = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]
    for value, freq in counts.items():
        buckets[freq].append(value)
    result = []
    for freq in range(len(buckets) - 1, 0, -1):
        for value in buckets[freq]:
            result.append(value)
            if len(result) == k:
                return result
    return result
```

Edge cases k equals the number of distinct values → return all of them.

Interviewer may ask next Solve it with a min-heap of size k. When is the heap better?

06 Course Schedule

Topological Sort (Kahn's BFS) Medium

Some courses must be taken before others. Given the prerequisite pairs, can you finish every course?

`EXAMPLE n = 2, prereqs = [[1,0],[0,1]] → False (they depend on each other)`

HOW TO THINK

1. Treat courses as nodes and prerequisites as arrows. A cycle makes it impossible.
2. Start with every course that has no prerequisites (in-degree 0) and 'take' it.
3. Taking a course lowers its dependents' in-degree; queue any that hit 0. If you take all n, there's no cycle.

COMPLEXITY

Time O(V + E)

Space O(V + E)

```
def can_finish(num_courses, prereqs):
    graph = defaultdict(list)
    indegree = [0] * num_courses
    for course, before in prereqs:
        graph[before].append(course)
        indegree[course] += 1
    ready = deque(c for c in range(num_courses) if indegree[c] == 0)
    done = 0
    while ready:
        c = ready.popleft()
        done += 1
        for nxt in graph[c]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                ready.append(nxt)
    return done == num_courses
```

Edge cases No prerequisites at all → True.

Interviewer may ask next Return a valid order to take the courses (Course Schedule II).

07 Clone Graph

Graph Traversal + Hash Map | Medium

Make a deep copy of a connected undirected graph: new nodes, same values, same connections.

EXAMPLE 1–2, 2–3, 3–4, 4–1 → a new graph with identical shape

HOW TO THINK

1. Keep a map from each original node to its copy so no node is copied twice (this also stops infinite loops on cycles).
2. Traverse with DFS or BFS; for every neighbour, create its copy if needed and connect it.

COMPLEXITY

Time O(V + E)

Space O(V)

```
class GraphNode:
    def __init__(self, val, neighbors=None):
        self.val = val
        self.neighbors = neighbors or []


def clone_graph(node):
    if node is None:
        return None
    copies = {node: GraphNode(node.val)}
    stack = [node]
    while stack:
        cur = stack.pop()
        for nb in cur.neighbors:
            if nb not in copies:
                copies[nb] = GraphNode(nb.val)
                stack.append(nb)
            copies[cur].neighbors.append(copies[nb])
    return copies[node]
```

Edge cases Empty graph (None) → None. A single node with no neighbours.

Interviewer may ask next How would you clone a linked list that also has a random pointer?

08 Number of Islands

Grid BFS / DFS · Medium

In a grid of '1' (land) and '0' (water), count the islands. Land connects up, down, left and right.

`EXAMPLE grid = ["11000","11000","00100","00011"] → 3`

HOW TO THINK

1. Scan every cell. When you hit unvisited land, that's a new island — count it.
2. Flood-fill from there (BFS queue) to mark the whole island as visited.

COMPLEXITY

Time O(rows × cols)

Space O(rows × cols)

```
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    seen = set()
    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1" and (r, c) not in seen:
                islands += 1
                seen.add((r, c))
                queue = deque([(r, c)])
                while queue:
                    x, y = queue.popleft()
                    for nx, ny in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                        if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == "1" and (nx,
ny) not in seen:
                            seen.add((nx, ny))
                            queue.append((nx, ny))
    return islands
```

Edge cases Empty grid → 0. All water → 0. One giant island → 1.

Interviewer may ask next Return the size of the largest island instead.

09 Lowest Common Ancestor in a BST

BST Ordering · Medium

In a binary search tree, find the deepest node that has both given nodes p and q in its subtree.

`EXAMPLE` tree rooted at 6, p = 2, q = 8 → 6

HOW TO THINK

1. If both values are smaller than the current node, the answer is on the left; if both are bigger, it's on the right.
2. The first node where they split (or that equals one of them) is the answer.

COMPLEXITY

Time O(height)

Space O(1)

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def lca_bst(root, p, q):
    node = root
    while node:
        if p.val < node.val and q.val < node.val:
            node = node.left
        elif p.val > node.val and q.val > node.val:
            node = node.right
        else:
            return node
    return None
```

Edge cases p is an ancestor of q → p itself is the answer.

Interviewer may ask next Solve it for a normal binary tree that is not a BST.

10 Level Order Traversal

BFS with a Queue · Medium

Return a binary tree's values level by level, from top to bottom.

`EXAMPLE` `tree [3,9,20,null,null,15,7] → [[3], [9, 20], [15, 7]]`

HOW TO THINK

1. Use a queue. At the start of each round, the queue holds exactly one level.
2. Pop that many nodes, record their values, and push their children for the next round.

COMPLEXITY

Time O(n)

Space O(width of tree)

```
def level_order(root):
    if root is None:
        return []
    levels, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        levels.append(level)
    return levels
```

Edge cases Empty tree → [].

Interviewer may ask next Print it in zig-zag order, or return only the rightmost value of each level.

11 Kth Largest Element

Min-Heap of Size k · Medium

Find the kth largest number in an unsorted array (not the kth distinct).

EXAMPLE `nums = [3,2,1,5,6,4], k = 2 → 5`

HOW TO THINK

1. Keep a min-heap holding the k largest values seen so far.
2. When the heap grows past k, remove its smallest. At the end, the top of the heap is the answer.

COMPLEXITY

Time O(n log k)

Space O(k)

```
def kth_largest(nums, k):
    smallest_of_top_k = []
    for x in nums:
        heapq.heappush(smallest_of_top_k, x)
        if len(smallest_of_top_k) > k:
            heapq.heappop(smallest_of_top_k)
    return smallest_of_top_k[0]
```

Edge cases Duplicates count separately: [3,3,2], k = 2 → 3.

Interviewer may ask next Quickselect does it in O(n) on average. Explain the trade-off.

12 Search in Rotated Sorted Array

Modified Binary Search · Medium

A sorted array was rotated at an unknown point (e.g. [4,5,6,7,0,1,2]). Find a target's index in O(log n), or return −1.

EXAMPLE `nums = [4,5,6,7,0,1,2], target = 0 → 4`

HOW TO THINK

1. At any midpoint, at least one half is still properly sorted.
2. Check whether the target lies inside that sorted half's range; if yes search there, otherwise search the other half.

COMPLEXITY

Time O(log n)

Space O(1)

```
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:                # left half is sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                                    # right half is sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Edge cases Array of size 1. Array that wasn't rotated at all.

Interviewer may ask next What if the array can contain duplicates?

13 Binary Search

Halving the Search Space · Easy

Find a target's index in a sorted array, or return −1.

`EXAMPLE` `nums = [-1,0,3,5,9,12], target = 9 → 4`

HOW TO THINK

1. Compare with the middle element and throw away the half that can't contain the target.
2. Use mid = lo + (hi − lo) // 2 — in languages with fixed-size integers this avoids overflow.

COMPLEXITY

Time O(log n)

Space O(1)

```
def binary_search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Edge cases Target smaller than everything or larger than everything. Empty array.

Interviewer may ask next Find the first and last position of a target that appears many times.

Level 2 · Linked Lists & Stacks

14 LRU Cache

Hash Map + Doubly Linked List | Medium / Hard

Design a fixed-size cache with get and put in O(1). When it's full, evict the least recently used key.

EXAMPLE capacity 2: put(1), put(2), get(1), put(3) → **key 2 is evicted**

HOW TO THINK

1. The hash map finds a node instantly; the doubly linked list keeps nodes in order of use.
2. On every get or put, move the node to the front. When over capacity, drop the node at the back.
3. Dummy head and tail nodes remove all the empty-list special cases.

COMPLEXITY

Time O(1) per operation

Space O(capacity)

```
class _Node:
    __slots__ = ("key", "val", "prev", "next")

    def __init__(self, key=0, val=0):
        self.key, self.val, self.prev, self.next = key, val, None, None


class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.head, self.tail = _Node(), _Node()   # head side = most recent
        self.head.next, self.tail.prev = self.tail, self.head

    def _unlink(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _push_front(self, node):
        node.prev, node.next = self.head, self.head.next
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.map:
            return -1
        node = self.map[key]
        self._unlink(node)
        self._push_front(node)
        return node.val

    def put(self, key, value):
        if key in self.map:
            self._unlink(self.map[key])
        node = _Node(key, value)
        self.map[key] = node
        self._push_front(node)
        if len(self.map) > self.cap:
            oldest = self.tail.prev
```

```
            self._unlink(oldest)
            del self.map[oldest.key]
```

Edge cases Updating an existing key must refresh it, not add a duplicate.

Interviewer may ask next Python's OrderedDict can do this in a few lines — but be ready to build it by hand.

15 Merge Two Sorted Lists

Dummy Node + Two Pointers · Easy

Merge two sorted linked lists into one sorted list by relinking their nodes.

EXAMPLE `[1,2,4] and [1,3,4] → [1,1,2,3,4,4]`

HOW TO THINK

1. Start from a dummy node so you never special-case the head.
2. Repeatedly attach the smaller front node; when one list runs out, attach the rest of the other.

COMPLEXITY

Time O(n + m)

Space O(1)

```
class ListNode:
    def __init__(self, val, next=None):
        self.val, self.next = val, next


def merge_two_lists(a, b):
    dummy = tail = ListNode(0)
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

Edge cases One or both lists empty.

Interviewer may ask next Merge k sorted lists (use a heap).

16 Reverse a Linked List

Pointer Rewiring · Easy

Reverse a singly linked list and return the new head.

EXAMPLE 1 → 2 → 3 → 4 → 5 → **5 → 4 → 3 → 2 → 1**

HOW TO THINK

1. Walk the list once, pointing each node's next back at the previous node.
2. Save the next node before you overwrite the pointer.

COMPLEXITY

Time O(n)

Space O(1)

```
def reverse_list(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev
```

Edge cases Empty list or a single node.

Interviewer may ask next Write it recursively, then reverse only the nodes between positions left and right.

17 Min Stack

Stack Storing Running Minimum · Medium

Design a stack that supports push, pop, top and get_min — all in O(1).

EXAMPLE push −2, 0, −3 → get_min, pop, get_min → **−3, then −2**

HOW TO THINK

1. Store each value together with the minimum of the stack at the moment it was pushed.
2. The top pair always knows the current minimum, even after pops.

COMPLEXITY

Time O(1) per operation

Space O(n)

```
class MinStack:
    def __init__(self):
        self.items = []                # (value, minimum so far)

    def push(self, x):
        current_min = min(x, self.items[-1][1]) if self.items else x
        self.items.append((x, current_min))

    def pop(self):
        self.items.pop()

    def top(self):
        return self.items[-1][0]

    def get_min(self):
        return self.items[-1][1]
```

Edge cases Duplicate minimum values must survive a pop correctly.

Interviewer may ask next Can you do it with O(1) extra space using an encoding trick?

18 Valid Parentheses

Stack Easy

Given a string of brackets ()[]{}, decide whether every bracket is closed in the right order.

`EXAMPLE "{[]}" and "(]" → True and False`

HOW TO THINK

1. Push opening brackets. On a closing bracket, the top of the stack must be its matching opener.
2. At the end the stack must be empty.

COMPLEXITY

Time O(n)

Space O(n)

```
def is_valid(s):
    match = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in match:
            if not stack or stack.pop() != match[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

Edge cases A lone closing bracket "]" or an unclosed "(".

Interviewer may ask next Find the length of the longest valid parentheses substring.

19 Merge Intervals

Sort + Sweep · Medium

Merge every overlapping interval and return the result.

EXAMPLE `[[1,3],[2,6],[8,10],[15,18]]` → `[[1,6],[8,10],[15,18]]`

HOW TO THINK

1. Sort by start time so overlapping intervals sit next to each other.
2. If an interval starts before the last merged one ends, extend that end; otherwise start a new block.

COMPLEXITY

Time O(n log n)

Space O(n)

```
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

Edge cases Touching intervals like [1,4] and [4,5] merge into [1,5].

Interviewer may ask next Insert a new interval into an already merged, sorted list.

20 3Sum

Sort + Two Pointers · Medium

Find all unique triplets in an array that add up to zero.

`EXAMPLE` `nums = [-1,0,1,2,-1,-4] → [[-1,-1,2], [-1,0,1]]`

HOW TO THINK

1. Sort. Fix one number, then find pairs for it with two pointers moving inwards.
2. Skip repeated values for the fixed number and the left pointer so no triplet is listed twice.

COMPLEXITY

Time $O(n^2)$

Space O(1) extra (besides output)

```
def three_sum(nums):
    nums = sorted(nums)
    triplets = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        lo, hi = i + 1, len(nums) - 1
        while lo < hi:
            total = nums[i] + nums[lo] + nums[hi]
            if total < 0:
                lo += 1
            elif total > 0:
                hi -= 1
            else:
                triplets.append([nums[i], nums[lo], nums[hi]])
                lo += 1
                while lo < hi and nums[lo] == nums[lo - 1]:
                    lo += 1
    return triplets
```

Edge cases All zeros → a single [0,0,0]. Fewer than 3 numbers → [].

Interviewer may ask next 3Sum Closest: return the sum nearest to a target.

21 Product of Array Except Self

Prefix & Suffix Products · Medium

For each position, return the product of every other element — without using division.

`EXAMPLE` `nums = [1,2,3,4] → [24,12,8,6]`

HOW TO THINK

1. Answer[i] = (product of everything to the left) × (product of everything to the right).
2. Fill left products in one pass, then multiply in right products in a reverse pass.

COMPLEXITY

Time O(n)

Space O(1) extra (besides output)

```
def product_except_self(nums):
    n = len(nums)
    answer = [1] * n
    left = 1
    for i in range(n):
        answer[i] = left
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= right
        right *= nums[i]
    return answer
```

Edge cases One zero makes every other position 0; two zeros make everything 0.

Interviewer may ask next Why is 'divide the total product' a bad idea here?

22 Longest Substring Without Repeats

Sliding Window · Medium

Find the length of the longest substring with no repeating characters.

EXAMPLE `s = "abcabcbb"` → **3 ("abc")**

HOW TO THINK

1. Grow a window to the right, remembering the last index of every character.
2. When a character repeats inside the window, jump the window's start just past its previous position.

COMPLEXITY

Time O(n)

Space O(size of alphabet)

```
def longest_unique_substring(s):
    last_seen = {}
    start = best = 0
    for i, ch in enumerate(s):
        if last_seen.get(ch, -1) >= start:
            start = last_seen[ch] + 1
        last_seen[ch] = i
        best = max(best, i - start + 1)
    return best
```

Edge cases Empty string → 0. "abba" checks that start never moves backwards.

Interviewer may ask next Allow at most k distinct characters instead.

23 Best Time to Buy and Sell Stock

Single Pass / Greedy · Easy

Given daily prices, choose one day to buy and a later day to sell for the biggest profit.

`EXAMPLE prices = [7,1,5,3,6,4] → 5 (buy at 1, sell at 6)`

HOW TO THINK

1. Track the cheapest price seen so far.
2. Each day, check the profit of selling today and keep the best.

COMPLEXITY

Time O(n)

Space O(1)

```
def max_profit(prices):
    cheapest = float("inf")
    profit = 0
    for price in prices:
        cheapest = min(cheapest, price)
        profit = max(profit, price - cheapest)
    return profit
```

Edge cases Prices only fall → profit 0.

Interviewer may ask next Allow unlimited buy/sell transactions.

24 Two Sum

Hash Map · Easy

Return the indices of the two numbers that add up to a target.

`EXAMPLE nums = [2,7,11,15], target = 9 → [0, 1]`

HOW TO THINK

1. For each number x, the partner you need is target − x.
2. Store numbers you've already seen in a map (value → index) and check for the partner before storing x.

COMPLEXITY

Time O(n)

Space O(n)

```
def two_sum(nums, target):
    index_of = {}
    for i, x in enumerate(nums):
        if target - x in index_of:
            return [index_of[target - x], i]
        index_of[x] = i
    return []
```

Edge cases Same value used twice, like [3,3] with target 6.

Interviewer may ask next If the array is sorted, solve it with two pointers and O(1) space.

★ BONUS

6 Questions Freshers Get Asked Again and Again

Short, classic, and a favourite in campus drives, service-company rounds and first technical screens. Make sure you can write each one from memory.

B1 Maximum Subarray (Kadane's Algorithm)

Running Best · Medium

Find the contiguous subarray with the largest sum.

EXAMPLE `nums = [-2,1,-3,4,-1,2,1,-5,4]` → `6 ([4,-1,2,1])`

HOW TO THINK

1. At each number, either extend the current subarray or start fresh from this number — whichever is bigger.
2. Track the best sum seen at any point.

COMPLEXITY

Time O(n)

Space O(1)

```
def max_subarray(nums):
    best = current = nums[0]
    for x in nums[1:]:
        current = max(x, current + x)
        best = max(best, current)
    return best
```

Edge cases All negative numbers → the largest single number.

Interviewer may ask next Also return the start and end index of that subarray.

B2 Valid Anagram

Character Counting · Easy

Check whether two strings use exactly the same letters the same number of times.

`EXAMPLE "anagram", "nagaram" → True`

HOW TO THINK

1. Different lengths → False straight away.
2. Otherwise compare letter counts (or sort both strings).

COMPLEXITY

Time O(n)

Space O(alphabet)

```
def is_anagram(a, b):
    return len(a) == len(b) and Counter(a) == Counter(b)
```

Edge cases Upper vs lower case and spaces — ask the interviewer whether they count.

Interviewer may ask next What if the input contains Unicode characters?

B3 Detect a Cycle in a Linked List

Fast & Slow Pointers · Easy

Does a linked list loop back on itself?

`EXAMPLE 3 → 2 → 0 → −4 → (back to 2) → True`

HOW TO THINK

1. Move one pointer one step and another two steps at a time.
2. If there's a loop, the fast pointer eventually laps the slow one; if not, it reaches the end.

COMPLEXITY

Time O(n)

Space O(1)

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False
```

Edge cases Empty list, single node, single node pointing to itself.

Interviewer may ask next Return the node where the cycle begins.

B4 Missing Number

Math / XOR · Easy

An array holds n distinct numbers from 0 to n with exactly one missing. Find it.

```
EXAMPLE  nums = [3,0,1] → 2
```

HOW TO THINK

1. The full sum 0 + 1 + ... + n is n(n+1)/2.
2. Subtract the array's sum — what's left is the missing number.

COMPLEXITY

Time O(n)

Space O(1)

```python
def missing_number(nums):
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)
```

Edge cases Missing number is 0 or n.

Interviewer may ask next Solve it with XOR to avoid any overflow concerns.

B5 Group Anagrams

Hash Map with a Signature Key · Medium

Group words that are anagrams of each other.

```
EXAMPLE  ["eat","tea","tan","ate","nat","bat"] →
[["eat","tea","ate"], ["tan","nat"], ["bat"]]
```

HOW TO THINK

1. Anagrams share the same sorted letters — use that as the map key.
2. Append each word to its key's list.

COMPLEXITY

Time O(n · k log k)

Space O(n · k)

```python
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())
```

Edge cases Empty strings group together.

Interviewer may ask next Use a 26-letter count tuple as the key to avoid sorting.

B6 Climbing Stairs

Fibonacci-style DP · Easy

You can climb 1 or 2 steps at a time. In how many ways can you reach step n?

EXAMPLE n = 5 → 8

HOW TO THINK

1. To land on step n you came from step n−1 or step n−2, so ways(n) = ways(n−1) + ways(n−2).
2. Keep only the last two counts.

COMPLEXITY

Time O(n)

Space O(1)

```
def climb_stairs(n):
    one_back, two_back = 1, 1          # ways to reach step i-1 and i-2
    for _ in range(n - 1):
        one_back, two_back = one_back + two_back, one_back
    return one_back
```

Edge cases n = 1 → 1 way.

Interviewer may ask next What if you can also take 3 steps, or some steps are broken?

BEFORE YOUR INTERVIEW

Revision checklist

Tick a problem only when you can explain the idea, write the code without help, test it on an edge case and state its complexity.

- [ ] 01 Trapping Rain Water
- [ ] 02 House Robber
- [ ] 03 Coin Change
- [ ] 04 Word Break
- [ ] 05 Top K Frequent Elements
- [ ] 06 Course Schedule
- [ ] 07 Clone Graph
- [ ] 08 Number of Islands
- [ ] 09 Lowest Common Ancestor in a BST
- [ ] 10 Level Order Traversal
- [ ] 11 Kth Largest Element
- [ ] 12 Search in Rotated Sorted Array
- [ ] 13 Binary Search
- [ ] 14 LRU Cache
- [ ] 15 Merge Two Sorted Lists
- [ ] 16 Reverse a Linked List
- [ ] 17 Min Stack
- [ ] 18 Valid Parentheses
- [ ] 19 Merge Intervals
- [ ] 20 3Sum
- [ ] 21 Product of Array Except Self
- [ ] 22 Longest Substring Without Repeats
- [ ] 23 Best Time to Buy and Sell Stock
- [ ] 24 Two Sum
- [ ] B1 Maximum Subarray (Kadane's Algorithm)
- [ ] B2 Valid Anagram
- [ ] B3 Detect a Cycle in a Linked List
- [ ] B4 Missing Number
- [ ] B5 Group Anagrams
- [ ] B6 Climbing Stairs

Talk while you code

Interviewers score how you think, not just the final answer. Silence is the most common reason good coders get rejected.

Start simple

A correct brute-force answer you can improve beats a half-finished “optimal” one.

Test before you say “done”

Run your code by hand on the example and on one edge case (empty input, one element, duplicates).

Know your complexity

Always finish with time and space complexity — it's asked in almost every round.

Practised the problems?
Now land the interview.

FirstOffer lists fresh fresher jobs, internships and off-campus drives every day — with direct HR emails, Google Forms and official apply links. No middlemen.

firstoffer.online

Daily job alerts on our WhatsApp Channel · Questions vary by company and role; practise patterns, not just answers.